

AgentRecBench

Benchmarking LLM Agent-based Personalized Recommender Systems

Joaquin Burdiles

Alonso Tamayo

Marcelo Vargas

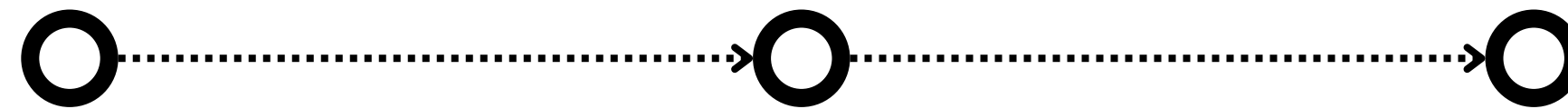
Contexto

¿Por que necesitamos algo nuevo?

Reglas manuales

Machine Learning

Deep Learning



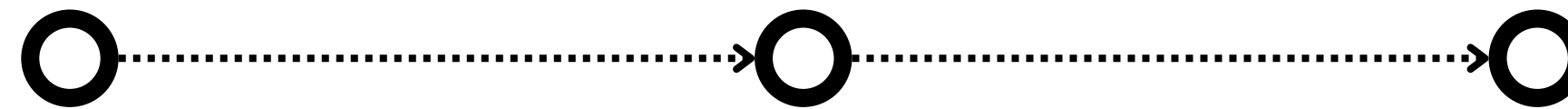
Contexto

¿Por que necesitamos algo nuevo?

Reglas manuales

Machine Learning

Deep Learning



Problemas críticos

01

Caja negra

No explica por qué recomienda

02

Depende de historial masivo

Falla con usuarios nuevos

03

Estrategias fijas

No se adapta a contextos distintos

Definición formal del problema | ¿Qué es y cómo funciona?

$$E = (U, I, H)$$

Usuarios, Items e Historial

$$a_t \sim \pi_{\theta}(a | s_t)$$

El agente observa y decide su acción

01

Observa

Perfil del usuario + contexto

02

Razona

¿Necesito más info o ya puedo recomendar?

03

Actúa

Recomienda un ítem o busca más datos

La evolución de los sistemas de recomendación

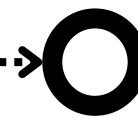
De modelos fijos a
agentes que razonan

Reglas manuales



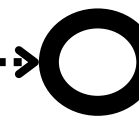
- Rígido y no escala
- No aprende
- Falla fuera de sus reglas

Deep Learning



- Aprende de patrones numéricos
- Caja negra, no explica
- Necesita muchos datos

Agentic



- Entiende texto y razona
- Explicable y adaptativo
- Funciona con pocos datos

Contribución del paper

¿Qué construye este paper?

"No existía ningún benchmark estándar para evaluar estos sistemas"

01 Entorno simulador textual

- Yelp
- Amazon
- Goodreads

02 Escenarios de evaluación

- Classic
- Evolving
- Cold-start

03 Framework modular

- Planning
- Reasoning
- Tools
- Memory

04 Benchmark comparando métodos

- Primer benchmark público
- Leaderboard activo
- 295 equipos compitieron

Datos utilizados

amazon

- Reseñas de usuarios
- Metadata de items
- Conexiones

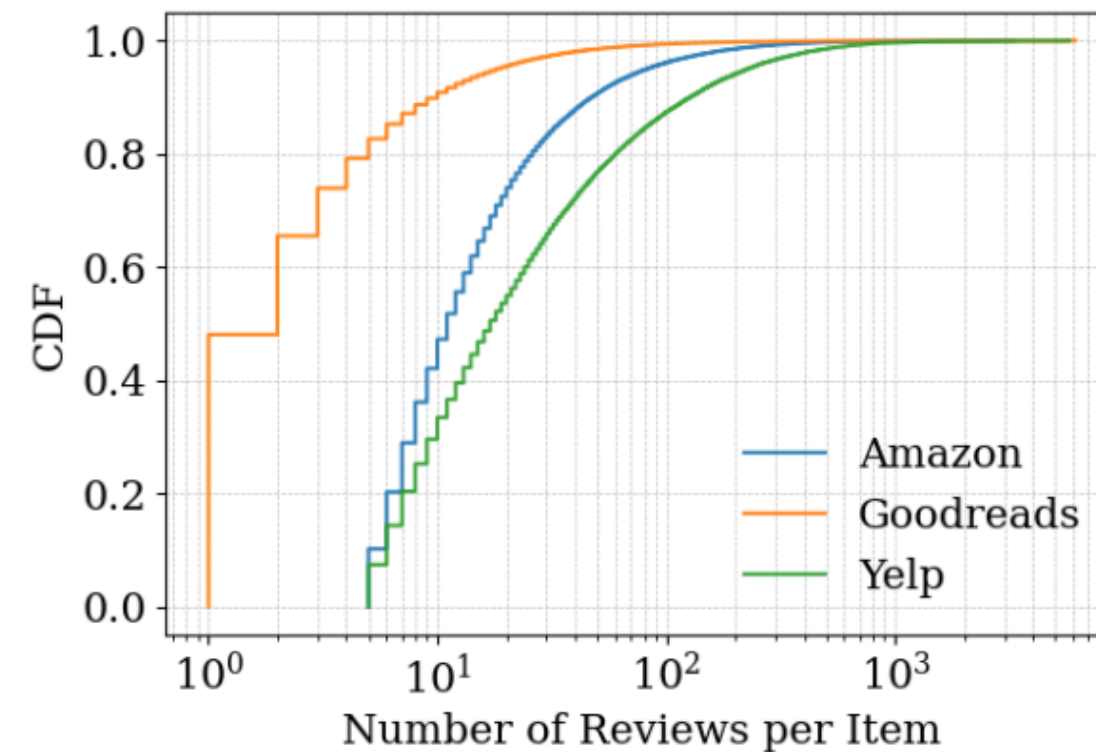
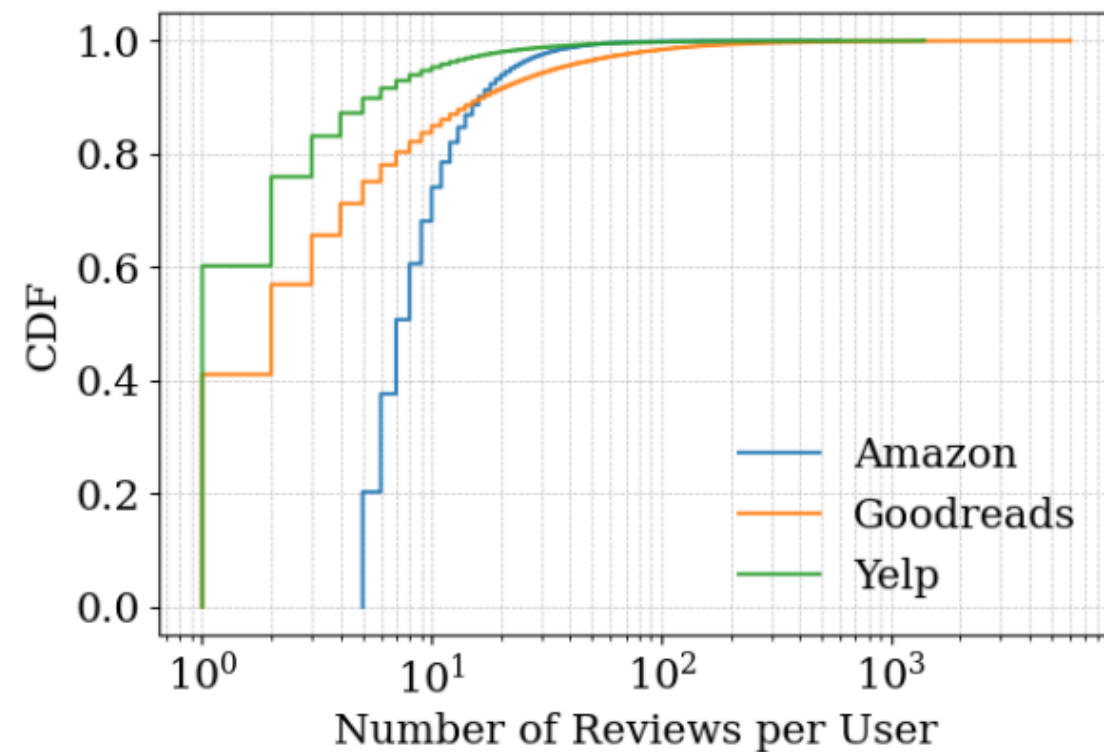
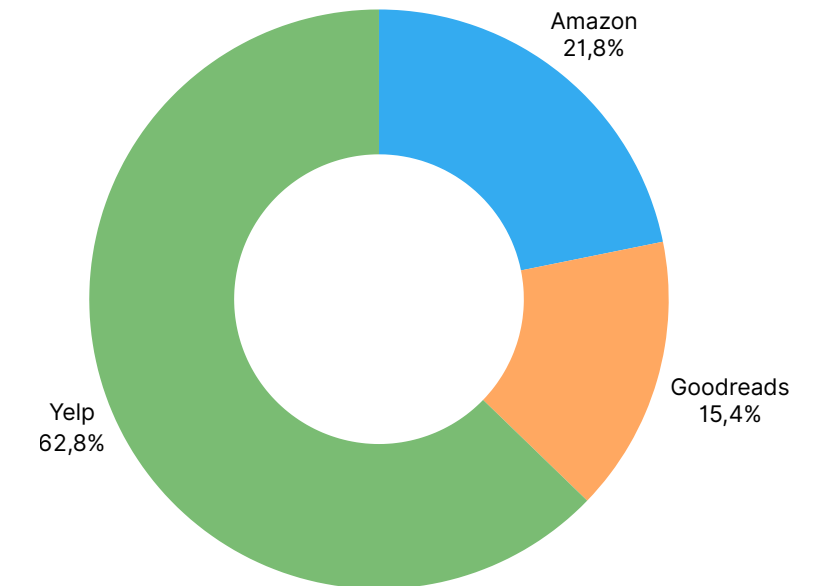
goodreads

- Reseñas de usuarios
- Metadata de libros
- Interacciones libro-usuario

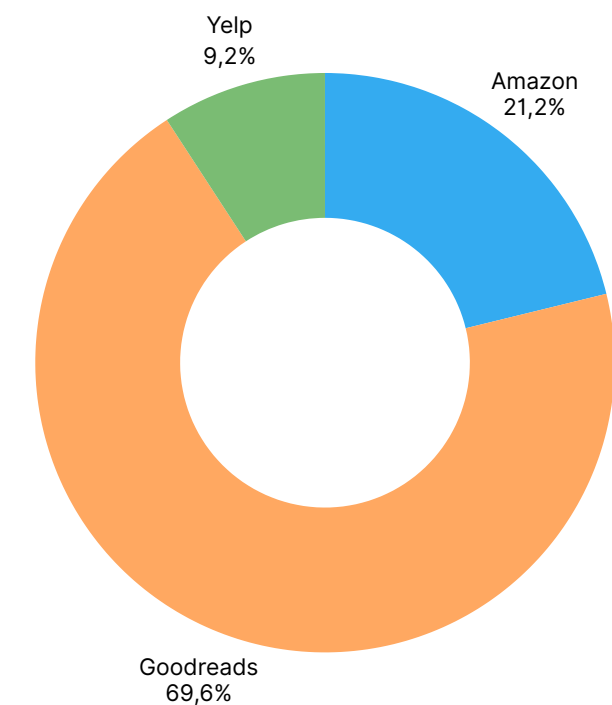
yelp

- Reseñas de usuarios
- Metadata de empresas
- Imágenes

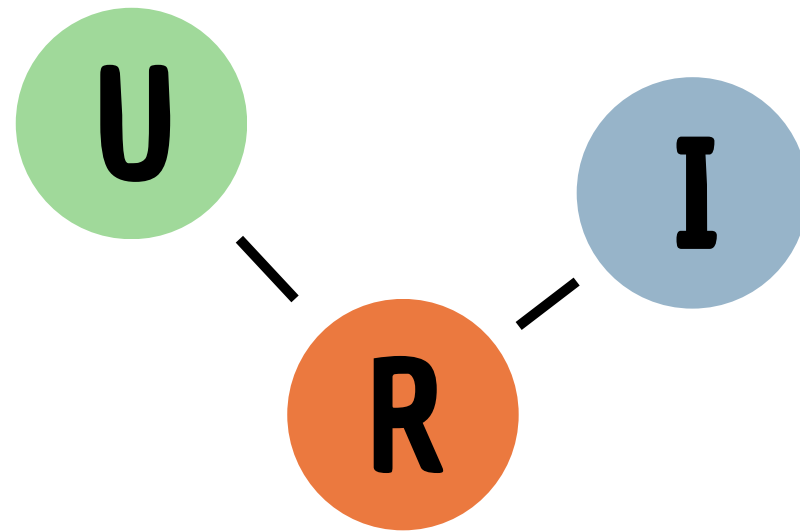
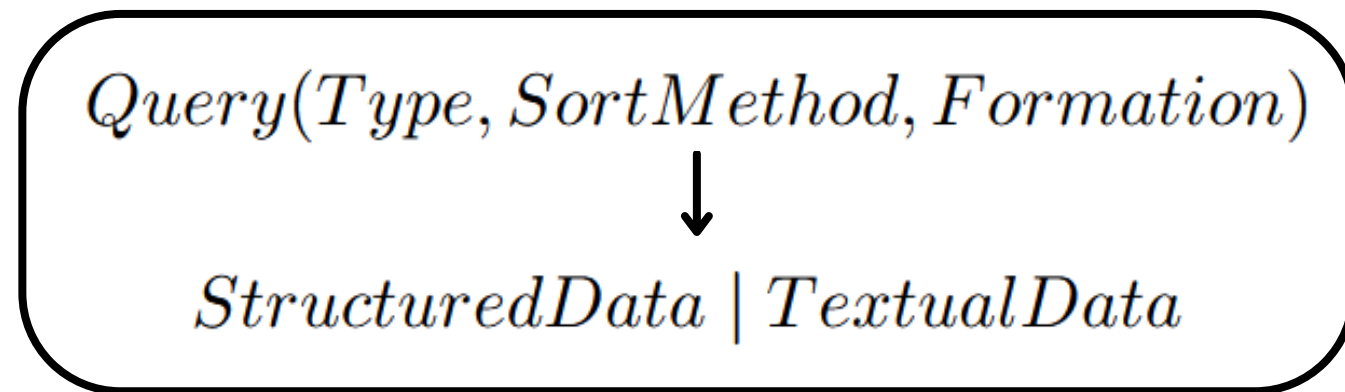
Distribución de usuarios



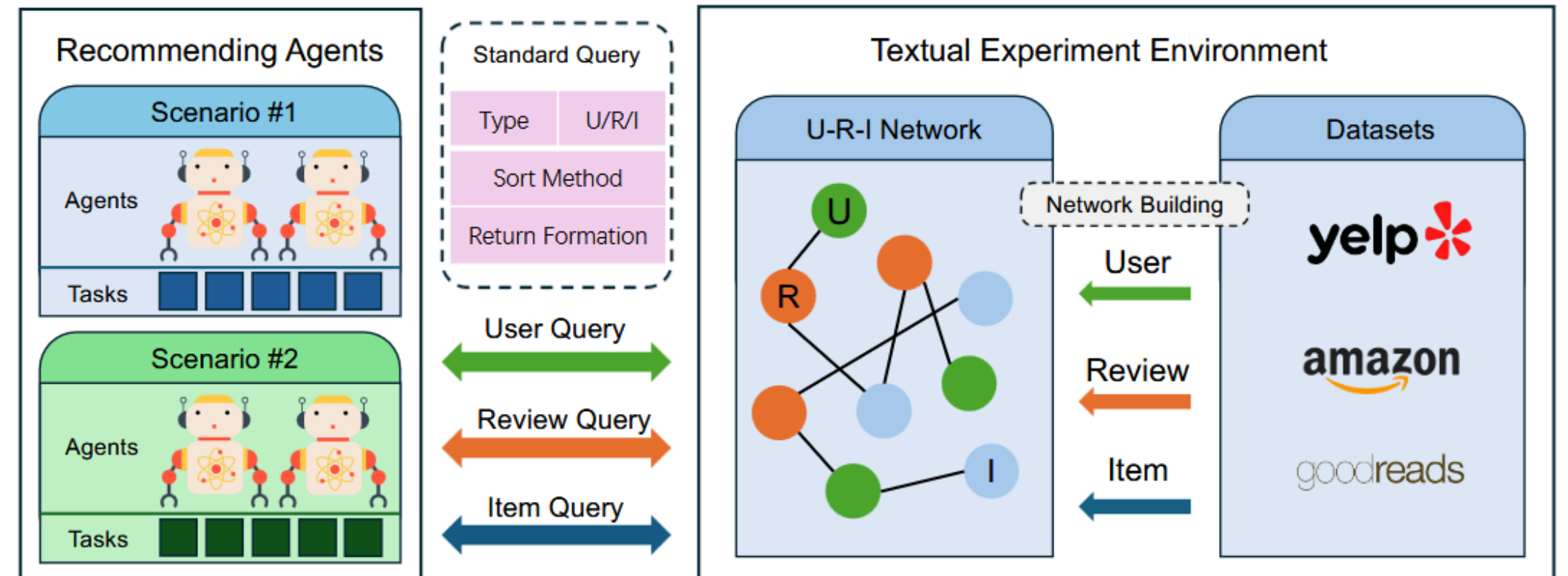
Distribución de items



Entorno simulador (U-R-I Network) User-Review-Item



Las conexiones entre los nodos
ilustran **interacciones**



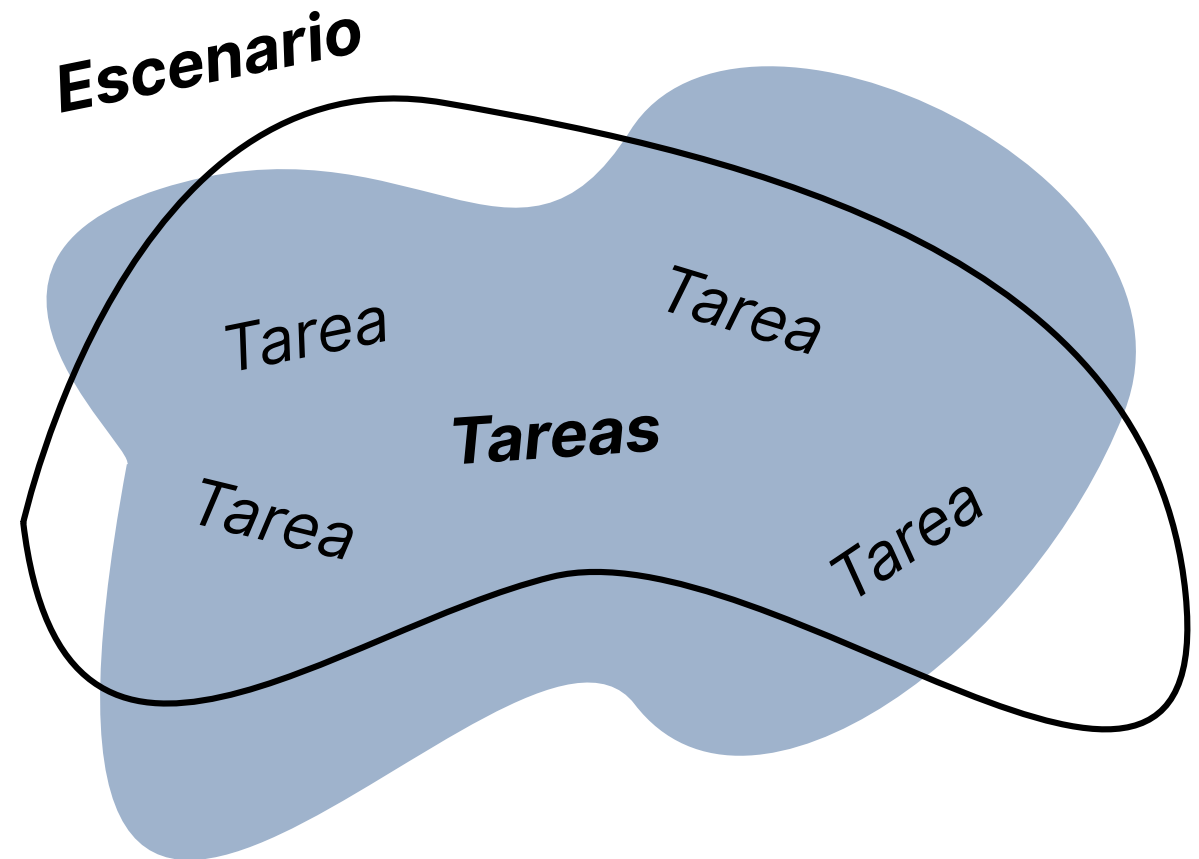
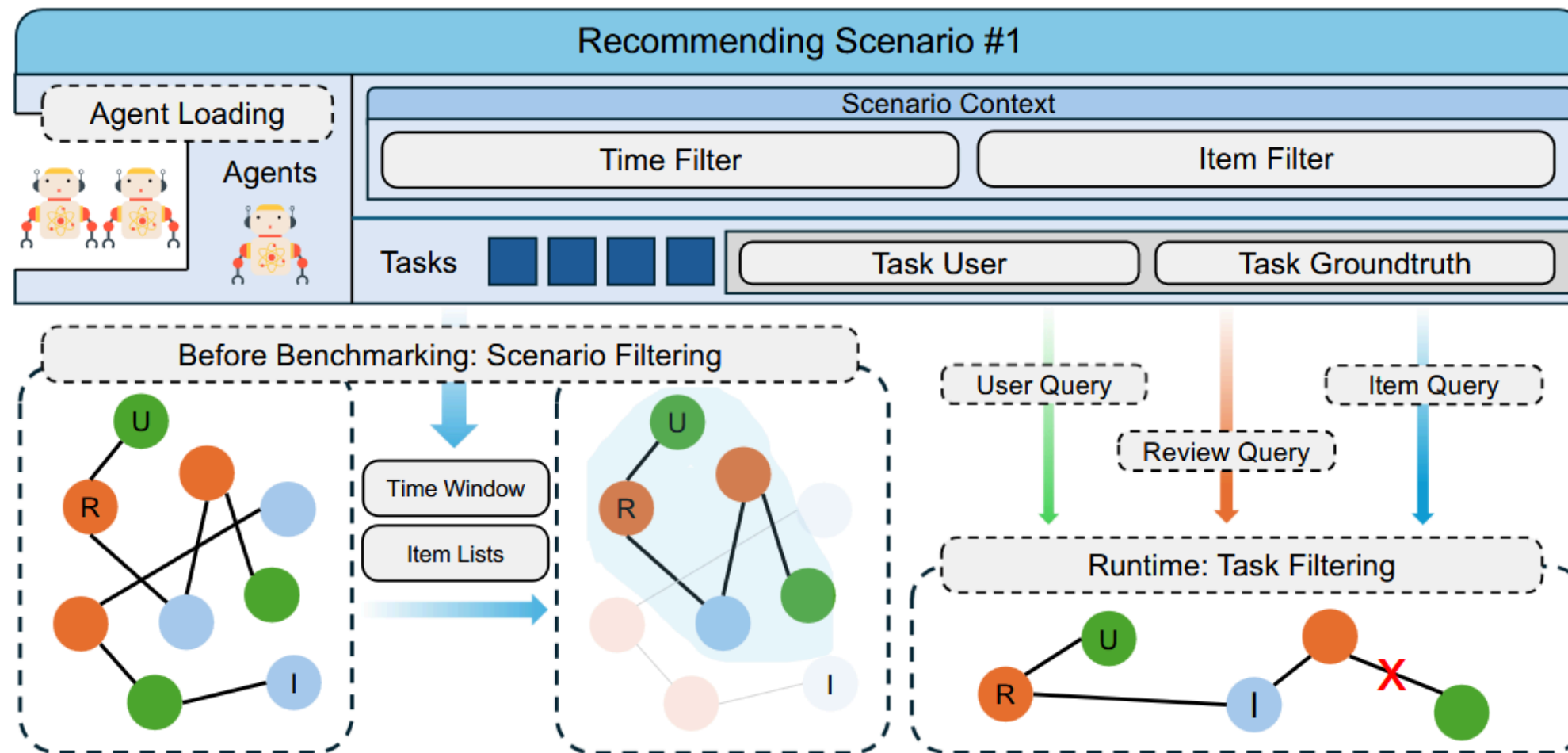
Type define el tipo de dato a recuperar (ej: usuario, item, reseña)
SortMethod define como son ordenados los resultados de la query
Formation indica el formato de los datos retornados

Control de visibilidad de datos

$Scenario = \{TimeFilter, ItemFilter\}$

TimeFilter define el alcance temporal de los datos

ItemFilter define el criterio de inclusión de items



$Task = \{TargetUser, GroundTruth\}$

TargetUser define el usuario al que estan destinadas las recomendaciones

GroundTruth entrega preferencia e interacciones pasadas del usuario

Escenarios de evaluación

CLASSIC

Evalúa el rendimiento general bajo condiciones estándar en las que los agentes tienen acceso completo al perfil e interacciones del usuario

EVOLVING-INTEREST

Evalúa la adaptación temporal en dos escalas: largo plazo (tres meses) para preferencias estables, y corto plazo (una semana) para patrones emergentes

COLD-START

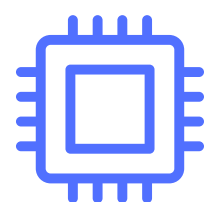
Evalúa la escasez de datos en usuarios e ítems con pocas interacciones. Mide la capacidad del sistema para utilizar información contextual y razonar sobre atributos

Agentes evaluados



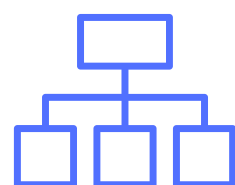
Sistemas tradicionales y Deep Learning

MF (basado en IDs y ratings numéricos) y LightGCN (grafos de interacciones)



Agentes base (Autores)

BaseAgent, CoTAgent, MemoryAgent y CoTMemAgent



Agente externo

Agent4Rec (complejo, con razonamiento multi-paso)

Baseline666's core workflow

```
You are a real user on an online platform. Your historical item review text
and stars are as follows: ['user_id', 'review_count', 'friends', 'stars'...]
Now you need to rank the following 20 items: [candidate item list]
'according to their match degree to your preference'.
Please rank the more interested items more front in your rank list.
The information of the above 20 candidate items is as follows: ['item_id',
'name', 'stars', 'review_count', 'attributes', 'title', 'average_rating',
'rating_number', 'description'...]
Your final output should be ONLY a ranked item list of [Candidate List]
with the following format!
DO NOT introduce any other item ids! DO NOT output your analysis process!
The correct output format: [Sorted Candidate Item List]
```



Agentes de Competencia (Top 3)

Baseline666 (1er lugar), RecHackers (2do) y DummyAgent (3er)

Métrica de evaluación

HR@N mide el porcentaje de veces que un ítem correcto logra quedar entre las primeras N recomendaciones

$$\text{HR@}N = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \mathbb{I}(p_t \in \mathcal{R}_t^N)$$

$\mathcal{T}$ es el set de prueba

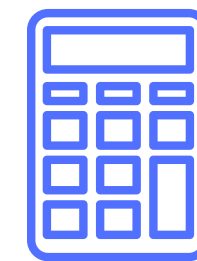
p_t es el ground-truth para el caso t

$\mathcal{R}_t^N$ denota el Top- N de recomendaciones

$\mathbb{I}(\cdot)$ es la función indicatriz



Cada agente recibe una lista de **20 candidatos**. Solo 1 es el correcto y 19 son distractores



El valor reportado es el **promedio** del rendimiento en el Top-1, Top-3 y Top-5

15%
Límite de azar

Resultados bajo esta cifra pueden considerarse **fracasos** del modelo.

Resultados

Classic
Recommendation

Escenario estándar: el agente recibe historial y candidatos; se evalúa ranking Top-N.

Método	Amazon	Goodreads	Yelp
MF	15	15	15
LightGCN	15	15	15
BaseAgent	39	15.7	3.7
Baseline666	69	45	7
RecHackers	54	45.3	7.7
Agent4Rec	23.3	9.3	5.7

69.0

Baseline666 en Amazon
≈ 4,6× sobre azar

15.0

Azar / colapso clásico
MF y LightGCN

7.7

Yelp más difícil
mejor caso Qwen

- Los métodos clásicos quedan en el valor aleatorio: no logran aprender por la alta dispersión.
- Los mejores agentes no son los más complejos: Baseline666 y RecHackers dominan el escenario clásico.
- La dificultad aumenta por dominio: Amazon es más favorable, Goodreads intermedio y Yelp el más desafiante.

Cold-start y Evolving Interest

Los resultados cambian cuando hay pocos datos o cuando los intereses evolucionan en el tiempo.

Método	Amazon U	Amazon I	Goodreads U	Goodreads I
Baseline666	48.7	48.3	36.7	44.1
DummyAgent	44.7	45.6	39.3	47.8
RecHackers	44.7	49.3	38.3	42.1
BaseAgent	16.6	15.0	16.0	21.2

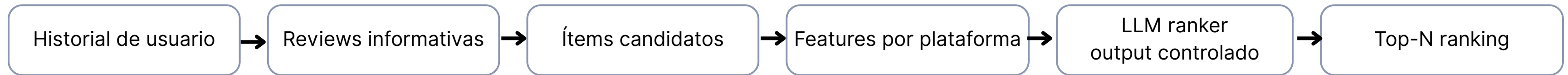
Método	Amazon LT	Amazon ST	Goodreads LT	Goodreads ST	Yelp ST
MF	38.4	49.5	17.7	21.3	65.9
LightGCN	32.3	59.1	18.0	22.3	68.9
Baseline666	50.6	55.3	63.0	55.7	0.0
RecHackers	52.7	57.3	62.0	54.0	5.3

- Los agentes de competencia siguen cerca de 45–49 en Amazon y Goodreads.
- Cold-start sigue siendo desafiante, pero los agentes aprovechan atributos y texto cuando falta historial.

- Cuando hay ventanas temporales suficientes, MF y LightGCN “reviven” y pueden superar agentes base.
- Yelp revela fragilidad: incluso Baseline666 cae a 0.0 en evolving, mostrando límites por dominio.

Lecciones aprendidas y diseño de agentes

El rendimiento mejora cuando el agente selecciona mejores señales, no cuando se agregan módulos sin estrategia.



Qué hicieron distinto los mejores agentes

1. Item-side features

Baseline666 adapta los atributos del ítem según el dominio, en vez de usar el mismo prompt para Amazon, Yelp y Goodreads.

2. Review-side features

DummyAgent y RecHackers enriquecen el perfil con reseñas útiles: texto, rating, fecha, votos, verificación y comentarios.

3. Prompt restringido

Los mejores workflows piden rankear solo los candidatos, evitan introducir ítems nuevos y fuerzan una salida limpia.

Lecciones aprendidas y diseño de agentes

Señales relevantes por plataforma

Plataforma	Features de ítem	Features de review
Amazon	id, nombre, estrellas, nº reviews, descripción	fecha, verified purchase, texto, rating
Yelp	id, nombre, estrellas, review count	texto + votos useful / cool / funny
Goodreads	autor, año, libros similares, rating	votos, comentarios, estado de lectura

Principios de diseño que deja el paper

Menos módulos ≠ peor agente

CoT y memoria no garantizan mejoras si no hay buena selección de contexto.

Contexto curado > historial completo

No basta con pasar muchas reviews, hay que priorizar las que aportan señal.

Dominio primero, arquitectura después

Un agente debe saber qué atributos importan en cada plataforma.

Complejidad con propósito

Agent4Rec es más sofisticado, pero suele rendir menos que workflows simples y bien diseñados.

Conclusiones y hallazgos clave

Conclusión general

Es el primer benchmark integral para evaluar sistemas de recomendación basados en agentes LLM.

Propone una plataforma común con entorno textual interactivo y evaluación reproducible.

Integra tres datasets reales (Amazon, Goodreads y Yelp), tres escenarios (classic, cold-start y evolving-interest) y compara más de 10 métodos.

Hallazgos clave

Los agentes LLM pueden superar métodos tradicionales cuando aprovechan mejor el contexto textual, atributos del ítem y reviews relevantes.

El resultado depende más del workflow que de la complejidad arquitectónica: selección de contexto, prompting y feature engineering por dominio.

Por eso Baseline666, DummyAgent y RecHackers rinden mejor que agentes más complejos pero peor diseñados.

Limitaciones y trabajo futuro

Limitaciones

Faltan más baselines clásicos y de deep learning

El entorno es solo textual

La evaluación actual es single-agent

Impacto

El benchmark ya mostró utilidad práctica con un challenge abierto y leaderboard público para investigación reproducible.

Trabajo futuro

Multimodalidad

Más datasets y dominios

Más métodos comparativos y evaluación de sistemas multi-agente

Referencias

- Schafer et al. (2007). Collaborative Filtering. *Springer*.
- Koren et al. (2009). Matrix Factorization. *IEEE Computer*.
- He et al. (2017). Neural Collaborative Filtering. *WWW*.
- He et al. (2020). LightGCN. *SIGIR*.
- Wang et al. (2023). RecMind. *arXiv:2308.14296*.